

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:

Студент 4 курса

Группы ИИ-26

Хомиченко И.А.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: Научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

Ход работы:

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
7	Fashion-MNIST	28X28	Adam

Код программы:

```
import torch

import torch.nn as nn

import torch.nn.functional as F

import torch.optim as optim

from torch.utils.data import DataLoader

import torchvision

import torchvision.transforms as transforms

import numpy as np

import matplotlib.pyplot as plt

import random

from PIL import Image

from collections import defaultdict
```

```
SEED = 42
```

```
random.seed(SEED)
```

```
np.random.seed(SEED)
```

```
torch.manual_seed(SEED)
```

```
if torch.cuda.is_available():
```

```
    torch.cuda.manual_seed_all(SEED)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
CLASSES = (
```

```
    "T-shirt/top",
```

```
    "Trouser",
```

```
    "Pullover",
```

```
    "Dress",
```

```
    "Coat",
```

```
    "Sandal",
```

```
    "Shirt",
```

```
    "Sneaker",
```

```
    "Bag",
```

```
    "Ankle boot"
```

```
)
```

```
MEAN = (0.2860,)
```

```
STD = (0.3530,)
```

```
train_transform = transforms.Compose([
```

```
transforms.RandomHorizontalFlip(),  
transforms.ToTensor(),  
transforms.Normalize(MEAN, STD)  
)
```

```
test_transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize(MEAN, STD)  
)
```

```
train_set = torchvision.datasets.FashionMNIST(  
    root="./data",  
    train=True,  
    download=True,  
    transform=train_transform  
)
```

```
test_set = torchvision.datasets.FashionMNIST(  
    root="./data",  
    train=False,  
    download=True,  
    transform=test_transform  
)
```

```
BATCH_SIZE = 128
```

```
train_loader = DataLoader(  
    train_set,  
    batch_size=BATCH_SIZE,  
    shuffle=True,  
    num_workers=0  
)
```

```
test_loader = DataLoader(  
    test_set,  
    batch_size=256,  
    shuffle=False,  
    num_workers=0  
)
```

```
image_shape = train_set[0][0].shape
```

```
print("Параметры программы")  
print(f"Устройство: {str(device).upper()}")  
print(f"Обучающая выборка: {len(train_set)} изображений")  
print(f"Тестовая выборка: {len(test_set)} изображений")  
print(f"Размер изображения: {image_shape[1]} x {image_shape[2]}")  
print(f"Количество каналов: {image_shape[0]}")  
print()
```

```
class FashionCNN(nn.Module):  
    def __init__(self, num_classes=10):
```

```
super().__init__()
```

```
self.conv1 = nn.Conv2d(  
    1,  
    32,  
    kernel_size=3,  
    padding=1  
)
```

```
self.conv2 = nn.Conv2d(  
    32,  
    64,  
    kernel_size=3,  
    padding=1  
)
```

```
self.conv3 = nn.Conv2d(  
    64,  
    128,  
    kernel_size=3,  
    padding=1  
)
```

```
self.pool = nn.MaxPool2d(  
    kernel_size=2,  
    stride=2  
)
```

```
self.fc1 = nn.Linear(  
    128 * 3 * 3,  
    256  
)
```

```
self.dropout = nn.Dropout(0.4)
```

```
self.fc2 = nn.Linear(  
    256,  
    num_classes  
)
```

```
def forward(self, x):  
    x = self.pool(F.relu(self.conv1(x)))  
    x = self.pool(F.relu(self.conv2(x)))  
    x = self.pool(F.relu(self.conv3(x)))  
  
    x = torch.flatten(x, 1)  
  
    x = F.relu(self.fc1(x))  
    x = self.dropout(x)  
    x = self.fc2(x)  
  
    return x
```

```
model = FashionCNN().to(device)
```

```
parameters_count = sum(  
    parameter.numel()  
    for parameter in model.parameters()  
    if parameter.requires_grad  
)
```

```
print("Архитектура нейронной сети")  
print("Вход 1x28x28")  
print("Conv2d 32")  
print("Conv2d 64")  
print("Conv2d 128")  
print("Полносвязный слой 256")  
print("Выход 10 классов")  
print(f"Обучаемых параметров: {parameters_count}")  
print()
```

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = optim.Adam(  
    model.parameters(),  
    lr=0.001  
)
```

```
N_EPOCHS = 8
```

```
history = {  
    "train_loss": [],
```

```
"train_acc": [],  
"test_loss": [],  
"test_acc": []  
}
```

```
def process_epoch(loader, training):
```

```
    if training:
```

```
        model.train()
```

```
    else:
```

```
        model.eval()
```

```
    loss_sum = 0.0
```

```
    correct_predictions = 0
```

```
    total_images = 0
```

```
    if training:
```

```
        context = torch.enable_grad()
```

```
    else:
```

```
        context = torch.no_grad()
```

```
    with context:
```

```
        for images, labels in loader:
```

```
            images = images.to(device)
```

```
            labels = labels.to(device)
```

```
        if training:
```

```
            optimizer.zero_grad()
```

```
outputs = model(images)

loss = criterion(outputs, labels)
```

```
if training:

    loss.backward()

    optimizer.step()
```

```
loss_sum += loss.item() * images.size(0)
```

```
predictions = outputs.argmax(dim=1)
```

```
correct_predictions += (

    predictions == labels

).sum().item()
```

```
total_images += labels.size(0)
```

```
average_loss = loss_sum / total_images

accuracy = correct_predictions / total_images
```

```
return average_loss, accuracy
```

```
print("Обучение нейронной сети")
```

```
for epoch in range(1, N_EPOCHS + 1):

    train_loss, train_acc = process_epoch(
```

```
train_loader,  
  
True  
  
)
```

```
test_loss, test_acc = process_epoch(  
  
    test_loader,  
  
    False  
  
)
```

```
history["train_loss"].append(train_loss)  
history["train_acc"].append(train_acc)
```

```
history["test_loss"].append(test_loss)  
history["test_acc"].append(test_acc)
```

```
print(  
    f"Эпоха {epoch} из {N_EPOCHS} "  
    f"Ошибка обучения {train_loss:.4f} "  
    f"Точность обучения {train_acc * 100:.2f}% "  
    f"Ошибка теста {test_loss:.4f} "  
    f"Точность теста {test_acc * 100:.2f}%"  
)
```

```
print()  
print("Результат обучения")  
print(  
    f"Итоговая точность на тестовой выборке: "
```

```
    f"{history['test_acc'][-1] * 100:.2f}%"  
)
```

```
epochs = range(1, N_EPOCHS + 1)
```

```
fig, axes = plt.subplots(  
    1,  
    2,  
    figsize=(13, 5)  
)
```

```
axes[0].plot(  
    epochs,  
    history["train_loss"],  
    marker="o",  
    markersize=3,  
    label="Обучение"  
)
```

```
axes[0].plot(  
    epochs,  
    history["test_loss"],  
    marker="o",  
    markersize=3,  
    label="Тест"  
)
```

```
axes[0].set_title("Ошибка модели")  
axes[0].set_xlabel("Эпоха")  
axes[0].set_ylabel("Ошибка")  
axes[0].legend()  
axes[0].grid(alpha=0.3)
```

```
axes[1].plot(  
    epochs,  
    [value * 100 for value in history["train_acc"]],  
    marker="o",  
    markersize=3,  
    label="Обучение"  
)
```

```
axes[1].plot(  
    epochs,  
    [value * 100 for value in history["test_acc"]],  
    marker="o",  
    markersize=3,  
    label="Тест"  
)
```

```
axes[1].set_title("Точность модели")  
axes[1].set_xlabel("Эпоха")  
axes[1].set_ylabel("Точность, %")  
axes[1].legend()  
axes[1].grid(alpha=0.3)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
model.eval()
```

```
class_correct = defaultdict(int)
```

```
class_total = defaultdict(int)
```

```
with torch.no_grad():
```

```
    for images, labels in test_loader:
```

```
        images = images.to(device)
```

```
        labels = labels.to(device)
```

```
        outputs = model(images)
```

```
        predictions = outputs.argmax(dim=1)
```

```
    for label, prediction in zip(labels, predictions):
```

```
        label_index = label.item()
```

```
        prediction_index = prediction.item()
```

```
        class_total[label_index] += 1
```

```
    if label_index == prediction_index:
```

```
        class_correct[label_index] += 1
```

```

print()

print("Точность распознавания классов")

for index, class_name in enumerate(CLASSES):

    accuracy = (

        class_correct[index]

        / class_total[index]

        * 100

    )

    print(

        f"{class_name}: {accuracy:.2f}%"

    )

def show_test_example(index=None):

    model.eval()

    if index is None:

        index = random.randrange(len(test_set))

    image, true_label = test_set[index]

    input_tensor = image.unsqueeze(0).to(device)

    with torch.no_grad():

        output = model(input_tensor)

```

```

probabilities = F.softmax(
    output,
    dim=1
).cpu().numpy().flatten()

predicted_label = int(
    np.argmax(probabilities)
)

display_image = image.squeeze().numpy()
display_image = display_image * STD[0] + MEAN[0]
display_image = np.clip(display_image, 0, 1)

fig, axes = plt.subplots(
    1,
    2,
    figsize=(11, 4),
    gridspec_kw={
        "width_ratios": [1, 1.7]
    }
)

axes[0].imshow(
    display_image,
    cmap="gray"
)

axes[0].axis("off")

```

```

if predicted_label == true_label:

    title_color = "green"
else:

    title_color = "red"


axes[0].set_title(

    f"Настоящий класс: {CLASSES[true_label]}\n"

    f"Ответ сети: {CLASSES[predicted_label]}",

    color=title_color

)


colors = []


for i in range(10):

    if i == true_label:

        colors.append("green")

    elif i == predicted_label:

        colors.append("red")

    else:

        colors.append("steelblue")


axes[1].barh(

    CLASSES,

    probabilities * 100,

    color=colors

)

```

```
axes[1].set_xlabel("Вероятность, %")  
axes[1].set_xlim(0, 100)  
axes[1].invert_yaxis()
```

```
plt.tight_layout()  
plt.show()
```

```
for _ in range(3):  
    show_test_example()
```

```
def predict_custom_image(path_or_pil):  
    model.eval()
```

```
    if isinstance(path_or_pil, str):  
        image = Image.open(  
            path_or_pil  
        ).convert("L")  
    else:  
        image = path_or_pil.convert("L")
```

```
preprocessing = transforms.Compose([  
    transforms.Resize((28, 28)),  
    transforms.ToTensor(),  
    transforms.Normalize(  
        MEAN,  
        STD
```

```
)  
])
```

```
input_tensor = (  
    preprocessing(image)  
    .unsqueeze(0)  
    .to(device)  
)
```

```
with torch.no_grad():  
    output = model(input_tensor)
```

```
probabilities = F.softmax(  
    output,  
    dim=1  
) .cpu().numpy().flatten()
```

```
predicted_label = int(  
    np.argmax(probabilities)  
)
```

```
fig, axes = plt.subplots(  
    1,  
    2,  
    figsize=(11, 4),  
    gridspec_kw={  
        "width_ratios": [1, 1.7]  
    }  
)
```

```
)
```

```
axes[0].imshow(  
    image,  
    cmap="gray"  
)
```

```
axes[0].axis("off")
```

```
axes[0].set_title(  
    f"Ответ сети: "  
    f"{CLASSES[predicted_label]}"  
)
```

```
colors = []
```

```
for i in range(10):  
    if i == predicted_label:  
        colors.append("red")  
    else:  
        colors.append("steelblue")
```

```
axes[1].barh(  
    CLASSES,  
    probabilities * 100,  
    color=colors  
)
```

```

axes[1].set_xlabel(
    "Вероятность, %"
)

axes[1].set_xlim(
    0,
    100
)

axes[1].invert_yaxis()

plt.tight_layout()
plt.show()

print()
print("Результат проверки своего изображения")
print(
    f"Определённый класс: "
    f"{CLASSES[predicted_label]}"
)
print(
    f"Уверенность модели: "
    f"{probabilities[predicted_label] * 100:.2f}%"
)

return predicted_label, probabilities

predict_custom_image(
    r"C:\Users\fhdgsjfbdjsg\Desktop\ОВиИС1\platye.jpg"
)

```

Результат тестирования:

Устройство: CPU
Обучающая выборка: 60000 изображений
Тестовая выборка: 10000 изображений
Размер изображения: 28 x 28
Количество каналов: 1

Архитектура нейронной сети
Вход 1x28x28
Conv2d 32
Conv2d 64
Conv2d 128
Полносвязный слой 256
Выход 10 классов
Обучаемых параметров: 390410

Обучение нейронной сети

Эпоха 1 из 8	Ошибка обучения 0.5989	Точность обучения 77.92%	Ошибка теста 0.3850	Точность теста 85.71%
Эпоха 2 из 8	Ошибка обучения 0.3590	Точность обучения 86.98%	Ошибка теста 0.3095	Точность теста 88.52%
Эпоха 3 из 8	Ошибка обучения 0.3018	Точность обучения 89.11%	Ошибка теста 0.3035	Точность теста 89.36%
Эпоха 4 из 8	Ошибка обучения 0.2741	Точность обучения 90.00%	Ошибка теста 0.2716	Точность теста 89.97%
Эпоха 5 из 8	Ошибка обучения 0.2527	Точность обучения 90.75%	Ошибка теста 0.2673	Точность теста 90.53%
Эпоха 6 из 8	Ошибка обучения 0.2345	Точность обучения 91.47%	Ошибка теста 0.2517	Точность теста 90.80%
Эпоха 7 из 8	Ошибка обучения 0.2169	Точность обучения 92.11%	Ошибка теста 0.2378	Точность теста 91.32%
Эпоха 8 из 8	Ошибка обучения 0.2069	Точность обучения 92.48%	Ошибка теста 0.2355	Точность теста 91.48%

Результат обучения

Итоговая точность на тестовой выборке: 91.48%

Точность распознавания классов

T-shirt/top: 89.30%

Trouser: 97.90%

Pullover: 89.20%

Dress: 91.00%

Coat: 87.70%

Sandal: 97.40%

Shirt: 70.40%

Sneaker: 97.40%

Bag: 98.90%

Ankle boot: 95.60%

Результат проверки своего изображения

Определённый класс: Bag

Уверенность модели: 97.00%

График изменения ошибки:

Figure 1

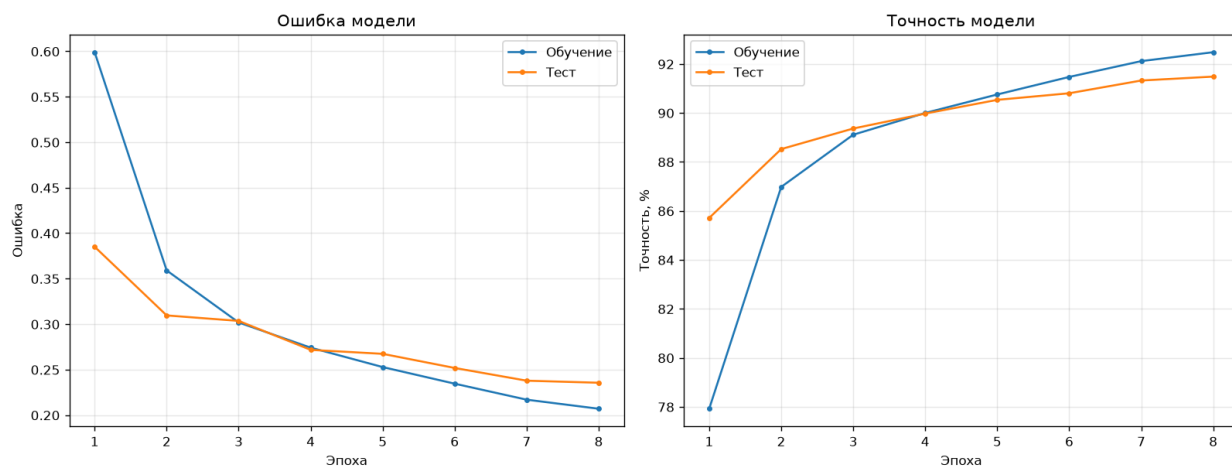


Figure 1

Настоящий класс: Sandal
 Ответ сети: Sandal

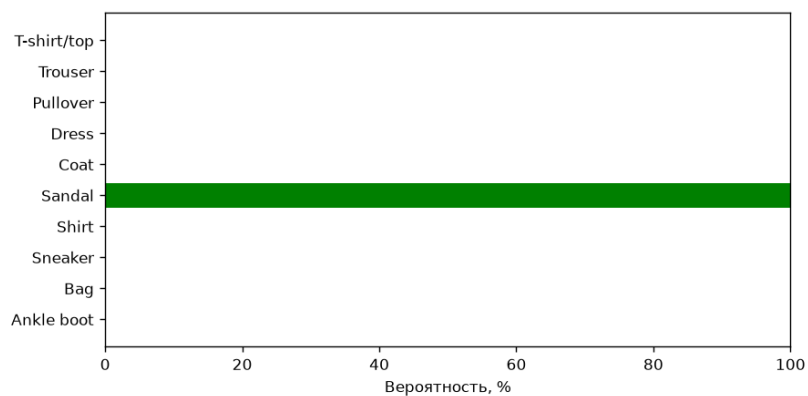
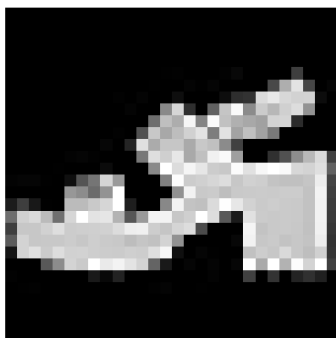
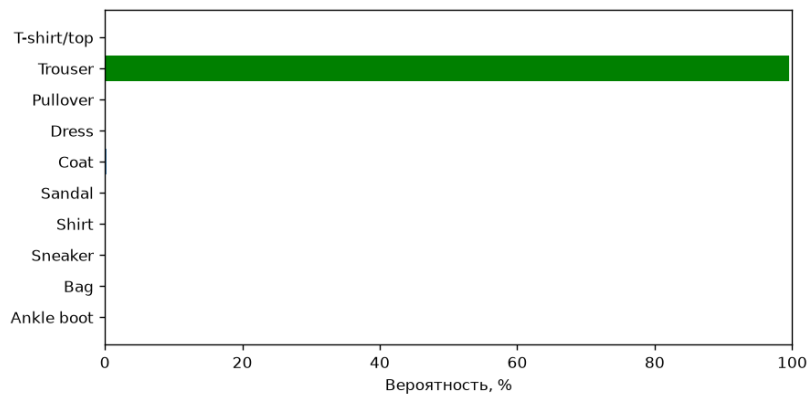
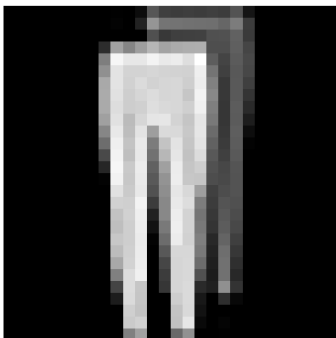
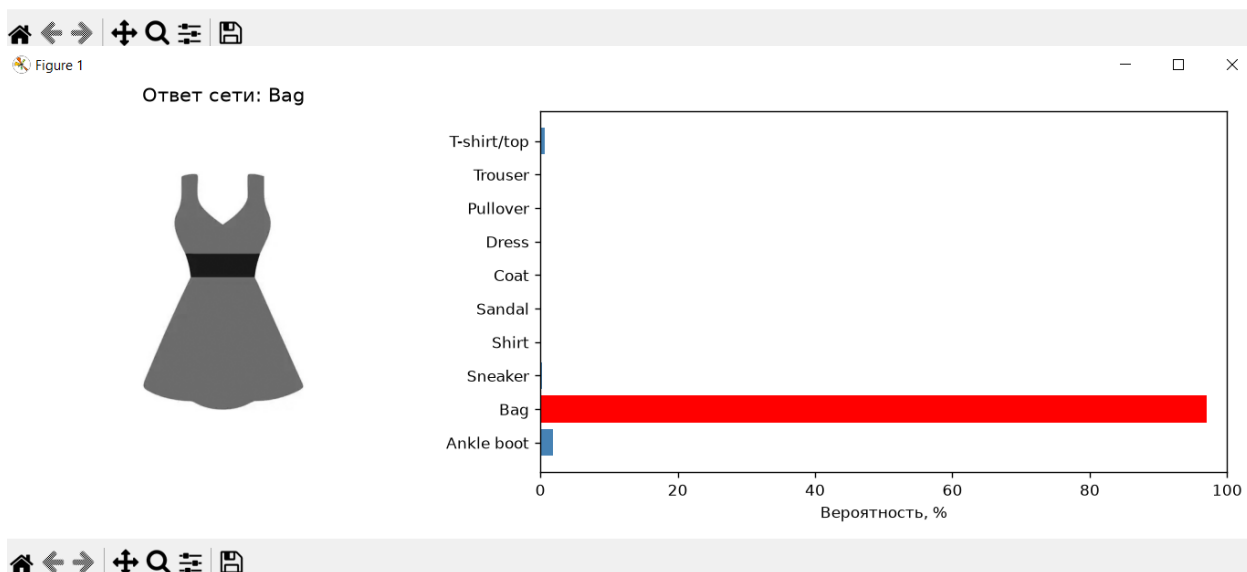
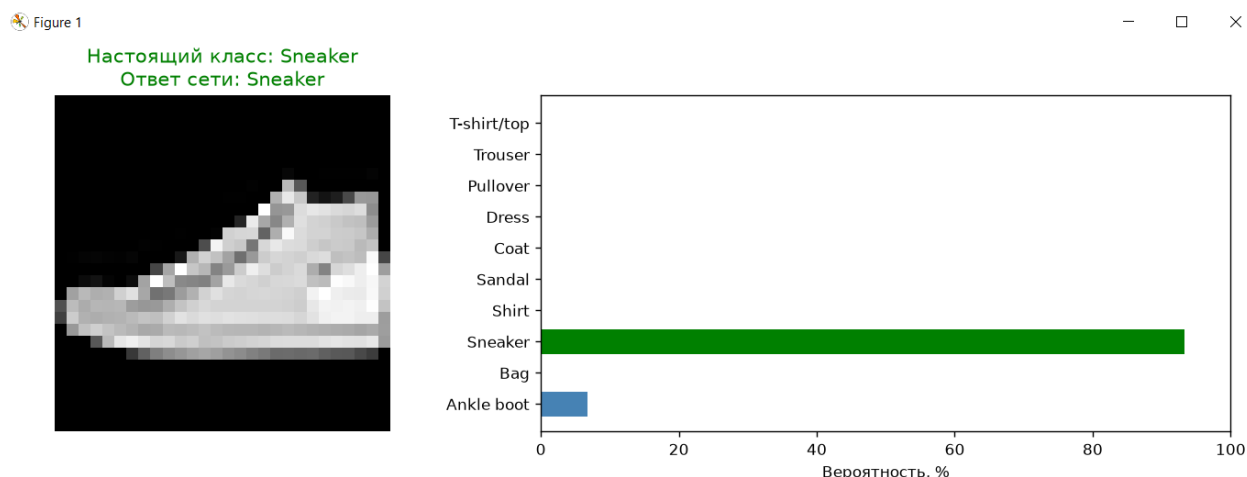


Figure 1

Настоящий класс: Trouser
 Ответ сети: Trouser





В ходе работы была обучена свёрточная нейронная сеть для классификации изображений из набора Fashion-MNIST. За 8 эпох точность модели на тестовой выборке достигла 91,48%, что показывает хорошую способность простой CNN выделять характерные признаки различных видов одежды и обуви. Лучше всего сеть распознаёт наиболее отличающиеся классы, такие как сумки, брюки и обувь, а наибольшие трудности возникают с визуально похожими объектами, например рубашками, футболками и верхней одеждой. При использовании обычных пользовательских фотографий качество классификации может снижаться, поскольку такие изображения отличаются от данных, использованных при обучении. Повысить точность можно за счёт усложнения архитектуры сети, улучшения предварительной обработки изображений и использования более разнообразных обучающих данных.

Вывод: Проведя данную лабораторную работу я научился конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения